

Comparative Evaluation of KV Cache Optimization Strategies for Efficient LLM Inference: Quantization, Sparse Attention, Eviction, and Latent Projection

Rishika Mamidibathula[†], Suhas Morisetty^{*}, Ziheng Wang^{*}, Tony Tian^{*}

^{*}Department of Computer Science, [†]Department of Data Science, Columbia University
COMS E6998: High Performance Machine Learning, Spring 2026
{rm4318, vm2825, zw3153, jt3640}@columbia.edu

Abstract—Autoregressive large-language-model decoding is bounded by key-value (KV) cache memory bandwidth: at batch size 32 and 4K context, Llama-2-7B requires roughly 65 GB for the cache alone, nearly saturating an H100 80 GB GPU. We benchmark five KV optimization techniques drawn from four compression families—per-token quantization (KIVI 2/4-bit), dynamic sparse selection (TopK / TokenSelect), one-shot eviction (SnapKV), and latent projection (TransMLA)—together with a quantized-sparse hybrid (KIVI×TopK), under a single harness that fixes model, hardware, decode loop, prompts, and seeds across runs. KIVI 4-bit preserves baseline LongBench quality and roughly doubles decode throughput at batch 32 by lifting the operational batch ceiling from 32 to 128. SnapKV slightly improves overall quality with no per-step decode cost; TopK $K = 1024$ matches baseline quality and gains substantially on retrieval-heavy tasks. MLA yields a consistent $3.9\times$ KV reduction even at batch 1, but the available checkpoint regresses on quality and requires retraining. The hybrid stacks memory savings of both compression axes at the cost of decode speed at long context. Each method occupies a distinct point in the quality–memory–latency space; the right choice depends on whether batch size, context length, or output fidelity is the binding constraint.

Index Terms—KV cache, LLM inference, quantization, sparse attention, token eviction, multi-head latent attention, throughput, memory efficiency

I. Introduction

LLM inference proceeds in two phases. Prefill processes the prompt in a single forward pass; decode then emits one token per step. Each decode step rereads every cached key and value tensor from HBM, making decode strongly memory-bandwidth bound rather than compute-bound. The cost grows linearly with sequence length, batch size, and depth. For Llama-2-7B [2] (32 layers, 32 heads of dimension 128, FP16) at 4096 tokens and batch size 32, the cache occupies $2 \times 32 \times 32 \times 128 \times 4096 \times 32 \times 2 \approx 65$ GB [1]—leaving little room for activations on an 80 GB GPU and pushing larger configurations into out-of-memory territory.

Research on KV reduction has converged on four principal compression families. Quantization reduces bits per element while keeping all tokens [3], [4]. Sparse selection attends to a dynamically chosen subset of tokens at each decode step [5], [6]. Eviction permanently discards low-importance tokens after prefill [8], [9], [10]. Latent projection stores a compressed low-rank representation instead of full K/V tensors [11], [12]. Cross-family comparison is difficult in practice because published evaluations rarely share the same model, hardware, or decode loop. This work addresses that gap with a unified harness whose only variable is the KV policy.

Our contributions are: (i) a single-codebase benchmark of five KV optimization techniques and a quantized-sparse hybrid on Llama-2-7B; (ii) a kernel-level analysis of crossover behavior—KIVI and TopK are slower than baseline at low batch but faster at high batch or long context; (iii) empirical evidence that KIVI 4-bit and SnapKV 0.4 deliver lossless or marginally better LongBench quality; and (iv) full reproducibility via Modal cloud scripts and a public W&B dashboard at <https://wandb.ai/rm4318-columbia-university/kv-cache-hpml>.

II. Literature Review

Quantization. KIVI [3] stores keys with per-channel scales (capturing channel-direction outliers) and values with per-token grouped quantization (group size 32), keeping the last 128 tokens in FP16 for short-range fidelity. At 2-bit it achieves roughly $8\times$ cache compression, at 4-bit roughly $4\times$. KVQuant [4] extends this with non-uniform codebooks and per-channel smoothing, reaching near-lossless 1-bit at greater overhead.

Sparse selection. TokenSelect [5]—the basis of our TopK implementation—retains the full FP16 cache but at each decode step selects only the top- K tokens for attention, fusing the scoring kernel in Triton. A cosine-similarity reuse cache avoids re-scoring when consecutive queries

are similar. RocketKV [6] adds a two-stage compression pipeline; RefreshKV [7] periodically refreshes evicted tokens.

Eviction. H2O [8] identifies “heavy-hitter” tokens by cumulative attention mass. SnapKV [9] performs a single eviction after prefill by voting over a small observation window, producing a fixed-size compressed cache with no per-step overhead. Ada-KV [10] allocates the budget adaptively per head.

Latent KV. DeepSeek-V2 [11] introduced multi-head latent attention (MLA), which stores a compressed latent vector c_t per token and expands to full K/V during attention via a learned up-projection, with Attention Absorption amortizing the projection cost. TransMLA [12] retrofits this structure onto an existing Llama-2 checkpoint.

III. Methodology

A. Model and Hardware

All experiments use meta-llama/Llama-2-7b-chat-hf (32 layers, 32 heads, head dim 128, FP16 weights) on a single NVIDIA H100 80 GB HBM3 GPU provided by Modal cloud compute. The runtime stack is pinned: PyTorch 2.4.1, HuggingFace Transformers 4.44.2, CUDA 12.1, Triton 3.0, flash-attn 2.5. The MLA experiments additionally use botsxc/llama2-7b-chat-mla-2048-8 [12] (2048-dim latent, 256-token calibration context, no further finetuning).

B. Datasets and Metrics

Quality. LongBench [14], six tasks $\times$ 20 examples = 120 examples per method: qasper (scientific QA, F1), multifielddqa_en (multi-document QA, F1), triviaqa (open-domain QA, F1), 2wikimqa (multi-hop QA, F1), multi_news (summarization, ROUGE-L), lcc (code completion, edit similarity). Inputs are middle-truncated to 4096 tokens following the KIVI paper convention.

Throughput. Synthetic fixed-size batches with prefill 1024 and generation 512 tokens, swept over batch sizes 1–256. Five batches per configuration; the first is dropped as warmup; the mean of the remaining four is reported.

Long context. Prefill 32,768 tokens, generation 512, batch size 1, used for TopK-Flash and the KIVI $\times$ TopK hybrid.

C. Profiling Tools

Memory: `torch.cuda.max_memory_allocated()` reset before each batch; this measures peak, not steady-state. Timing: `torch.cuda.synchronize() + time.perf_counter()` brackets prefill (TTFT, time to first token) and the decode loop (throughput in tokens per second over steps 2 to G). Operator-level traces are captured with `torch.profiler` and exported to Chrome trace. All runs are logged to Weights & Biases.

D. Method Implementations

All methods implement a shared MethodWrapper interface in methods/ and run through an identical generate loop in benchmark/runner.py.

Baseline (FP16): dense attention with no KV modification.

KIVI [3]. Keys are quantized per-channel to 2 or 4 bits (grouped across the sequence dimension per head); values are quantized per-token with group size 32. The last 128 tokens are kept in FP16 (the residual buffer). Quantization fuses INT4 packing in a Triton kernel (`triton_quantize_and_pack`); GEMV dequantizes on-the-fly via `cuda_bmm_fA_qB_outer`. The cache is stored as a 9-tuple per layer (quantized blocks + scales + zeros for K and V plus the FP16 residual).

TopK / TokenSelect [5]. The full FP16 cache is retained. Three-region attention budget: a sink window (first 128 tokens), dynamically-selected top- K middle tokens, and a local window (last 512 tokens). Per-decode-step scoring fuses $\mathbf{QK}^\top$ across all 32 heads in a Triton kernel (`_topk_score_kernel`) followed by softmax, per-head sum, and `torch.topk`. A cosine-similarity cache (threshold 0.9) skips re-scoring when consecutive queries are similar; RoPE correction passes `true_seq_length` to each step.

TopK-Flash. The selection logic above on a paged KV pool (256-token pages) plus `flash_attn_with_kvcache` for zero-copy selective attention. This avoids the 52 GB contiguous allocation that standard flash-attn requires at 32K context.

SnapKV [9]. After prefill, attention scores from the last 32 query positions per head (the observation window) are averaged, max-pooled with kernel size 7 to smooth noise, and the top $\lfloor S \times 0.4 \rfloor$ tokens per head are retained permanently. Decode runs over the fixed reduced cache with no per-step overhead.

MLA (TransMLA [12]). Each token stores a compressed latent $c_t \in \mathbb{R}^{2048}$ ($4\times$ compression vs. $d_k \cdot n_h = 4096$). At decode time, c_t is up-projected to full K/V by learned matrices; Attention Absorption folds the up-projection into the QKV computation.

KIVI $\times$ TopK Hybrid. Prefill uses `F.scaled_dot_product_attention` (flash-attn backed, $O(T)$ memory) to avoid 32K OOM. Decode scores all tokens via `cuda_bmm_fA_qB_outer` over INT4-packed K, selects $n_{\text{sink}} + K + n_{\text{local}}$ positions, and gathers the corresponding INT4 V rows into a contiguous buffer fed directly into `cuda_bmm_fA_qB_outer`—no intermediate FP16 V is materialized.

IV. Experimental Results

A. LongBench Quality

Table I reports per-task and overall LongBench scores. KIVI 4-bit and TopK $K=1024$ both achieve 0.292 (+0.3% and +0.4% over baseline 0.291)—within evaluation noise for 20 examples per task but consistent across runs. SnapKV 0.4 reaches 0.295 (+0.3%).

TABLE I

LongBench quality (F1 / ROUGE-L / edit-sim), 6 tasks $\times$ 20 ex.
Tasks: (1) qasper, (2) mfqa, (3) triviaqa, (4) 2wikimqa, (5) mnews,
(6) lcc.

Method	(1)	(2)	(3)	(4)	(5)	(6)	Avg
Baseline (FP16)	.172	.380	.644	.083	.181	.287	.291
KIVI 4-bit	.194	.360	.643	.083	.188	.286	.292
KIVI 2-bit	.133	.382	.608	.050	.191	.277	.273
TopK $K=2048$	.145	.329	.731	.075	.178	.282	.290
TopK $K=1024$	.153	.281	.773	.089	.183	.273	.292
TopK $K=512$	.145	.237	.751	.050	.178	.281	.274
SnapKV 0.4	.153	.387	.687	.083	.181	.277	.295
MLA (B=1)*	.036	.101	.064	.043	.103	.140	.081

baseline truncated to 2048 tokens vs. 4096 for others.

TABLE II

Decode throughput (tok/s) and peak GPU memory vs. batch size.
Prefill 1024, gen 512. “—” = OOM.

BS	Decode tok/s			Peak mem (GB)		
	Base	KIVI-2b	Spdup	Base	KIVI-2b	MLA
1	70	30	0.43 $\times$	15.1	14.0	13.8
2	135	61	0.45 $\times$	16.7	14.4	—
4	242	123	0.51 $\times$	20.0	15.3	—
8	351	243	0.69 $\times$	26.4	17.1	—
16	438	480	1.10 $\times$	39.3	20.7	—
32	497	957	1.93 $\times$	65.0	27.8	—
64	—	1295	∞	—	42.1	—
128	—	1517	∞	—	70.7	—

Per-task patterns reveal two opposite breakdown mechanisms when compression is too aggressive. KIVI 2-bit and TopK $K=512$ both end up near 0.274 overall, but for different reasons. KIVI’s quantization noise hurts qasper (−22%) and 2wikimqa (−40%), tasks that require precise recall of sparse mid-context facts. TopK $K=512$ hurts multifielqqa_en (−38%) because answer spans deeper than $n_{\text{sink}} + n_{\text{local}}$ are pruned. Conversely, TopK beats baseline on TriviaQA across all K (+13pt to +20pt): the answer appears in multiple paraphrases, so sparse selection drops distracting copies and concentrates attention. SnapKV’s one-shot vote actively aligns selection with answer-bearing positions, boosting triviaqa (+4pt) and multifielqqa (+0.7pt) while only hurting qasper (−2pt) where evidence spans fall outside the 32-token observation window.

B. Throughput and Memory

Table II shows the KIVI throughput sweep at prefill 1024, generation 512.

Crossover at BS=16. KIVI 2-bit decode throughput is 0.4–0.7 $\times$ baseline at BS 1–8 but crosses over at BS=16 (480 vs. 438 tok/s, 1.10 $\times$) and reaches 1.93 $\times$ at BS=32. The mechanism is straightforward: at small batch the decode $\mathbf{QK}^T$ is a GEMV, and INT4 GEMV does not map to H100 tensor-core units—it is roughly 2.5 $\times$ slower than FP16 GEMV. At large batch the same operation becomes a near-GEMM where the HBM bandwidth saving

TABLE III

Long-context results. Prefill 32,768 tokens, gen 512, BS=1.

Method	TTFT (ms)	Spdup	Peak mem	Saving
Baseline (FP16)	2627	1.00 $\times$	52.1 GB	—
TopK-Flash	1761	1.49 $\times$	42.1 GB	−19%
SnapKV 0.4	—	—	26.0 GB [†]	−50%
KIVI $\times$ TopK 4b	—	0.61 $\times$	25.8 GB	−50%

Baseline OOMs at 64K; TopK-Flash succeeds at 65.8 GB. [†]SnapKV peak memory at 32K is interpolated from short-context measurements; 32K SnapKV throughput not directly benchmarked.

(the cache is 2.3 $\times$ smaller at BS=32: 27.8 vs. 65.0 GB) outweighs dequantization cost.

Capacity extension. Baseline OOMs at BS=64; KIVI 2-bit reaches BS=128 at 70.7 GB and achieves 1,517 tok/s—about 3 $\times$ the maximum ever achievable with the baseline on this GPU. This is the strongest practical argument for KIVI: it raises the per-GPU concurrency ceiling, not just speed at fixed batch.

TTFT overhead. KIVI’s TTFT is 18–66% worse across batch sizes (998 vs. 1659 ms at BS=32) because the full KV cache is quantized and packed at the end of prefill—a cost proportional to $B \times S$. For latency-sensitive interactive workloads this is a real penalty; for throughput-oriented offline inference it is amortized over hundreds of decode steps.

MLA at B=1. At gen=256, MLA achieves 43–44 tok/s vs. 34 tok/s baseline (1.26–1.32 $\times$) across prefill lengths 512–2048. Even single-request decode is partially HBM-bandwidth bound: the 3.9 $\times$ KV reduction (e.g., 1207 $\rightarrow$ 311 MB at $p=2048$) reduces per-step HBM traffic proportionally. The speedup narrows at gen=512 because the MLA up-projection cost grows with output length.

C. Long-Context Results (32K)

TopK-Flash. At 32K context the 256-token paged KV pool eliminates the 52 GB contiguous allocation that standard flash-attn requires, reducing TTFT by 33% and peak memory by 19%. The win is not from sparsity but from allocation and memory-access-pattern improvement. Baseline OOMs at 64K; TopK-Flash succeeds at 65.8 GB.

KIVI $\times$ TopK hybrid. Combining INT4 storage with sparse attention cuts peak memory 50% (52.1 $\rightarrow$ 25.8 GB) but decode throughput drops 39% (21.0 $\rightarrow$ 12.9 tok/s). The slowdown has two sources: (a) $\mathbf{QK}^T$ must still score all 32K tokens every step before top- K can be applied—the scoring cost is identical to baseline, plus topk + sort + gather; (b) the INT4 V GEMV inherits the same low-batch dequantization penalty as KIVI. LongBench quality remains at 0.297, within noise of 0.291. The hybrid shows that orthogonal compression strategies stack memory savings, but a selection-cost reduction is needed before the latency win materializes.

TABLE IV
Deployment fit summary.

Method	KV ratio	Quality	Best regime
KIVI 4-bit	4×	Lossless (.292)	Batch serving $\geq$ BS=16
KIVI 2-bit	8×	−6.2% (.273)	Max-batch, accept no Qe Limitations
TopK $K=1024$	1× [†]	Lossless (.292)	Single-req, long context
SnapKV 0.4	2.5×	+0.3% (.295)	Any; quality-neutral
MLA (B=1)	3.9×	−65%* needs retrain	After finetuning at $\geq 7B$

[†]TopK keeps full FP16 cache; only attention scope is sparse.

*Truncation mismatch may inflate gap.

V. Discussion

A. Deployment Recommendations

Table IV summarizes the operating point of each method.

KIVI 4-bit is the best practical default for high-throughput batch inference: it costs nothing in quality, raises max batch from 32 to 128, and roughly triples peak single-GPU throughput. The cost is TTFT latency (+18–66%), which matters for interactive applications.

SnapKV 0.4 is the right choice when quality must be preserved and per-step overhead must be zero. Its 2.5× KV reduction has no decode-time cost because the compressed cache is built once after prefill.

TopK $K=1024$ is suited to single-request, long-context use ($> 32K$) where the paged variant (TopK-Flash) provides real TTFT and memory gains. At short context and low batch, scoring overhead dominates the saved attention work.

MLA shows the theoretical ceiling of latent projection: 3.9× KV reduction and 1.3× decode speedup are repeatable benefits even at B=1, but production-quality output requires a checkpoint trained at longer context with higher latent rank.

B. Kernel-Level Observations

INT4 GEMV inefficiency. `cuda_bmm_fa_qB_outer` performs on-the-fly INT4 $\rightarrow$ FP16 dequantization per GEMV. H100 tensor cores target INT8/FP16 GEMM; INT4 GEMV falls back to scalar or vector-INT4 paths. Bandwidth saving only exceeds this overhead when decode approaches GEMM at BS $\geq$ 16.

KIVI residual buffer hot path. At short generation lengths ($< \text{residual_length} \times n_{\text{layers}}$), nearly all attention concentrates in the FP16 residual; KIVI provides minimal storage benefit in this regime. This is also why KIVI TTFT is roughly 3× slower: the full prefill quantization is paid upfront.

Triton fused scoring. `_topk_score_kernel` fuses $\mathbf{QK}^T$ + softmax + head-sum into a single launch, reducing Python-dispatch overhead 2–3× vs. sequential PyTorch ops. At 1K–4K sequence lengths the critical path remains HBM bandwidth, not compute or launch latency.

Hybrid stays in INT4. The hybrid keeps data in INT4 across the gather $\rightarrow$ attention boundary

by passing the gathered packed tensor directly into `cuda_bmm_fa_qB_outer`, avoiding an FP16 V materialization that would have cost an extra allocation and HBM round-trip per layer per step.

Single architecture. Results are specific to Llama-2-7B with multi-head attention (32 heads, 32 KV heads). Models with grouped-query attention (Llama-3, Mistral) have fewer KV heads, which changes the compression profile.

Small evaluation set. LongBench results use 20 examples per task. Some per-task deltas (e.g., qasper KIVI 2-bit −22%) may partially reflect small- n variance, although the direction is consistent across tasks.

MLA checkpoint. The available TransMLA checkpoint was calibrated at 256-token context without task finetuning; a properly trained checkpoint would likely recover much of the 65% quality gap.

Hybrid scoring cost. The 50% memory win comes with a 39% decode penalty because scoring must precede selection. Hierarchical or lazy-scoring approaches could decouple them.

VI. Conclusion

We conducted a controlled benchmark of five KV-cache optimization methods spanning four compression families on Llama-2-7B. KIVI 4-bit provides lossless quality with 4× KV compression and 1.93× throughput at BS=32—our recommended default for batch-oriented serving. SnapKV marginally improves LongBench overall score with zero runtime overhead. TopK $K=1024$ matches baseline quality and gains substantially on retrieval-heavy tasks. MLA delivers a consistent 3.9× KV reduction and 1.3× decode speedup at B=1 but requires a properly trained checkpoint for production quality. The KIVI×TopK hybrid stacks memory savings of both axes but pays a decode penalty until scoring cost can be reduced. No single method dominates all axes; practitioners should select based on whether batch size, context length, or output fidelity is the binding constraint.

Future work should address: (i) batched decode for TopK to amortize the per-step scoring cost and produce a KIVI-style crossover at BS $\geq$ 8; (ii) MLA retraining at 4K–8K context with rank 16–32 to preserve 3.9× compression while recovering QA quality; (iii) hierarchical or lazy scoring in the KIVI×TopK hybrid to decouple the memory win from the decode penalty; and (iv) cross-method composition, e.g., SnapKV eviction followed by KIVI quantization on the retained tokens to stack savings from orthogonal axes.

Appendix

AI Use Disclosure

Per the HPML AI Use Policy. Team members used Claude (Anthropic) and GitHub Copilot for: debugging

CUDA OOM errors during KIVI kernel integration, clarifying Triton kernel semantics, and polishing report prose drafted by team members. All experimental numbers were produced by running our own Modal scripts on H100 containers; profiler-trace interpretations were verified against raw torch.profiler traces in results/. AI-suggested code was rewritten in our own words and re-validated to produce identical outputs to a hand-written reference. No analysis, interpretation, or written argument in this report was generated end-to-end by an AI tool.

References

- [1] J. Jiang, P. Yang, R. Zhang, and F. Liu, “Towards efficient large language model serving: A survey on system-aware KV cache optimization,” TechRxiv, 2026.
- [2] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” arXiv:2307.09288, 2023.
- [3] Z. Liu et al., “KIVI: A tuning-free asymmetric 2-bit quantization for KV cache,” in Proc. ICML, 2024.
- [4] C. Hooper et al., “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in NeurIPS, 2024.
- [5] W. Wu et al., “TokenSelect: Efficient long-context inference via dynamic token-level KV cache selection,” in Proc. EMNLP, 2025.
- [6] P. Behnam et al., “RocketKV: Accelerating long-context LLM inference via two-stage KV cache compression,” in Proc. ICML, 2025.
- [7] F. Xu, T. Goyal, and E. Choi, “RefreshKV: Updating small KV cache during long-form generation,” in Proc. ACL, 2025.
- [8] Z. Zhang et al., “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in NeurIPS, 2023.
- [9] Y. Li et al., “SnapKV: LLM knows what you are looking for before generation,” in NeurIPS, 2024.
- [10] Y. Feng et al., “Ada-KV: Optimizing KV cache eviction by adaptive budget allocation,” in NeurIPS, 2025.
- [11] DeepSeek-AI, “DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model,” arXiv:2405.04434, 2024.
- [12] Y. He et al., “TransMLA: Multi-head latent attention is all you need,” arXiv:2502.07864, 2025.
- [13] C.-C. Chang et al., “xKV: Cross-layer SVD for KV-cache compression,” arXiv:2503.18893, 2025.
- [14] Y. Bai et al., “LongBench: A bilingual, multitask benchmark for long context understanding,” arXiv:2308.14508, 2023.